

Replit Implementation Brief: Business Mode (Fully Coded)

Replit, this document contains the exact HTML, CSS, and JavaScript required to build the new **Business Mode** tier. Do not guess the implementation — use the code provided below to build the Brand Voice Vault, the Workflow Packs grid, and the dynamic Prompt Variable execution UI.

1. HTML Structure (Inject into the Left Column)

Add this HTML structure to the main app container. It should be hidden by default and only shown when the "Business Mode" tab is active.

HTML

```
<!-- Business Mode Container -->
<div id="pmg-business-mode" class="pmg-suite-container" style="display: none;">

  <!-- Brand Voice Vault -->
  <div class="biz-brand-vault">
    <h3>🏛️ Your Brand Voice Vault</h3>
    <p>Set this once. Every prompt will automatically use these rules.</p>

    <div class="biz-input-group">
      <label for="biz-audience">Target Audience (Who are you talking to?)</label>
      <input type="text" id="biz-audience" placeholder="e.g., Busy moms, B2B Sales">
    </div>

    <div class="biz-input-group">
      <label for="biz-tone">Brand Tone (How do you sound?)</label>
      <input type="text" id="biz-tone" placeholder="e.g., Professional but witty">
    </div>

    <div class="biz-input-group">
      <label for="biz-negative">Negative Constraints (What should the AI NEVER do?)</label>
      <textarea id="biz-negative" placeholder="e.g., Never use emojis, avoid cliche">
    </div>

    <button id="biz-save-brand" class="pmg-btn-secondary">Save Brand Voice</button>
    <span id="biz-save-status" style="display:none; color: #4ade80; margin-left: 10px;">Saved</span>
  </div>

  <!-- Workflow Packs Grid -->
  <div id="biz-packs-container">
    <h3>Workflow Packs</h3>
```

```

<div class="biz-grid">
  <button class="biz-pack-btn" data-pack="entrepreneur">🏢 Entrepreneur & B
  <button class="biz-pack-btn" data-pack="ecommerce">🛒 E-Commerce & Shopif
  <button class="biz-pack-btn" data-pack="builder">💻 Builder (Replit/Bolt)
  <button class="biz-pack-btn" data-pack="creator">🎬 Creator (YouTube/Soci
  <button class="biz-pack-btn" data-pack="marketing">📣 Marketing & Ads</bu
  <button class="biz-pack-btn" data-pack="cx">🎧 Customer Experience</butto
</div>
</div>

<!-- Dynamic Execution UI (Hidden until a template is selected) -->
<div id="biz-execution-ui" style="display: none;">
  <button id="biz-back-btn" class="pmg-btn-ghost">← Back to Packs</button>
  <h3 id="biz-template-title">Template Title</h3>
  <p id="biz-template-desc">Template description goes here.</p>

  <div id="biz-variables-container">
    <!-- Dynamic inputs will be injected here by JS -->
  </div>

  <button id="biz-generate-btn" class="pmg-btn-primary">✨ Generate Master Pr
</div>

</div>

```

2. CSS Styling

Add these styles to the main stylesheet to handle the grid layout and premium feel of Business Mode.

CSS

```

/* Business Mode Styles */
.biz-brand-vault {
  background: rgba(255, 255, 255, 0.03);
  border: 1px solid rgba(255, 255, 255, 0.1);
  border-radius: 12px;
  padding: 20px;
  margin-bottom: 30px;
}

.biz-input-group {
  margin-bottom: 15px;
}

.biz-input-group label {
  display: block;

```

```
font-size: 12px;
color: #a0aec0;
margin-bottom: 5px;
text-transform: uppercase;
letter-spacing: 0.5px;
}

.biz-input-group input,
.biz-input-group textarea {
width: 100%;
background: rgba(0, 0, 0, 0.2);
border: 1px solid rgba(255, 255, 255, 0.1);
color: #fff;
padding: 10px 12px;
border-radius: 8px;
font-family: inherit;
}

.biz-input-group textarea {
min-height: 80px;
resize: vertical;
}

.biz-grid {
display: grid;
grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
gap: 15px;
margin-bottom: 30px;
}

.biz-pack-btn {
background: rgba(255, 255, 255, 0.05);
border: 1px solid rgba(255, 255, 255, 0.1);
padding: 20px;
border-radius: 12px;
color: #fff;
font-weight: 600;
text-align: left;
cursor: pointer;
transition: all 0.2s ease;
}

.biz-pack-btn:hover {
background: rgba(74, 222, 128, 0.1);
border-color: #4ade80;
transform: translateY(-2px);
}
```

```
#biz-execution-ui {
  background: rgba(0, 0, 0, 0.2);
  border: 1px solid rgba(74, 222, 128, 0.3);
  border-radius: 12px;
  padding: 24px;
}

#biz-variables-container {
  margin: 20px 0;
}
```

3. JavaScript Logic

Add this logic to handle the Brand Vault saving, the dynamic variable extraction, and the final prompt assembly.

JavaScript

```
// 1. Template Database (Example with 2 templates, expand as needed)
const bizTemplates = {
  ecommerce: [
    {
      id: "shopify-desc",
      title: "Shopify Product Description",
      desc: "SEO-optimized, benefit-driven product copy.",
      template: "Write an SEO-optimized Shopify product description for [PRODUCT_NAME]"
    }
  ],
  entrepreneur: [
    {
      id: "cold-email",
      title: "Cold Email Sequence",
      desc: "A 3-part sequence to book demos.",
      template: "Write a 3-part cold email sequence targeting [DECISION_MAKER_TITLE]"
    }
  ]
};

let currentActiveTemplate = null;

// 2. Brand Vault Logic (Save/Load from localStorage)
function initBrandVault() {
  const savedAudience = localStorage.getItem('bizAudience');
  const savedTone = localStorage.getItem('bizTone');
  const savedNegative = localStorage.getItem('bizNegative');

  if (savedAudience) document.getElementById('biz-audience').value = savedAudience;
```

```

if (savedTone) document.getElementById('biz-tone').value = savedTone;
if (savedNegative) document.getElementById('biz-negative').value = savedNegat

document.getElementById('biz-save-brand').addEventListener('click', () => {
  localStorage.setItem('bizAudience', document.getElementById('biz-audience')
  localStorage.setItem('bizTone', document.getElementById('biz-tone').value);
  localStorage.setItem('bizNegative', document.getElementById('biz-negative')

  const status = document.getElementById('biz-save-status');
  status.style.display = 'inline';
  setTimeout(() => status.style.display = 'none', 2000);
});
}

// 3. Dynamic Execution UI Logic
function loadTemplateExecution(templateObj) {
  currentActiveTemplate = templateObj;

  document.getElementById('biz-packs-container').style.display = 'none';
  document.getElementById('biz-execution-ui').style.display = 'block';

  document.getElementById('biz-template-title').innerText = templateObj.title;
  document.getElementById('biz-template-desc').innerText = templateObj.desc;

  const varsContainer = document.getElementById('biz-variables-container');
  varsContainer.innerHTML = ''; // Clear previous

  // Extract variables like [PRODUCT_NAME] using regex
  const varRegex = /\[(.*?)\]/g;
  let match;
  const variables = [];

  while ((match = varRegex.exec(templateObj.template)) !== null) {
    if (!variables.includes(match[1])) variables.push(match[1]);
  }

  // Build input fields for each variable
  variables.forEach(v => {
    const cleanLabel = v.replace(/_/g, ' ');
    const html = `
      <div class="biz-input-group">
        <label>${cleanLabel}</label>
        <input type="text" class="biz-var-input" data-var="${v}" placeholder="E
      </div>
    `;
    varsContainer.insertAdjacentHTML('beforeend', html);
  });
}

```

```

// 4. Master Prompt Assembly
document.getElementById('biz-generate-btn').addEventListener('click', () => {
  if (!currentActiveTemplate) return;

  let finalPrompt = currentActiveTemplate.template;

  // Replace variables with user input
  const inputs = document.querySelectorAll('.biz-var-input');
  inputs.forEach(input => {
    const varName = input.getAttribute('data-var');
    const val = input.value || `[${varName}]`; // Fallback if empty
    finalPrompt = finalPrompt.replace(new RegExp(`\\[${varName}\\]`, 'g'), val);
  });

  // Append Brand Voice Vault rules
  const audience = document.getElementById('biz-audience').value;
  const tone = document.getElementById('biz-tone').value;
  const negative = document.getElementById('biz-negative').value;

  let brandRules = "\n\n--- BRAND VOICE RULES ---\n";
  if (audience) brandRules += `Target Audience: ${audience}\n`;
  if (tone) brandRules += `Tone of Voice: ${tone}\n`;
  if (negative) brandRules += `STRICT NEGATIVE CONSTRAINTS (Do NOT do this): ${`

  if (audience || tone || negative) {
    finalPrompt += brandRules;
  }

  // Send to Right Column Output (Integrate with existing PMG output logic)
  // Example: document.getElementById('pmgv3-output-box').value = finalPrompt;
  console.log("Master Prompt Generated:", finalPrompt);

  // Trigger the existing Analyze/Generate flow here
});

// Initialize
document.addEventListener('DOMContentLoaded', () => {
  initBrandVault();

  // Wire up back button
  document.getElementById('biz-back-btn').addEventListener('click', () => {
    document.getElementById('biz-execution-ui').style.display = 'none';
    document.getElementById('biz-packs-container').style.display = 'block';
  });

  // Example: Wire up the first pack button for testing
  document.querySelector('[data-pack="ecommerce"]').addEventListener('click', (

```

```
loadTemplateExecution(bizTemplates.ecommerce[0]);  
});  
});
```